



ClearPath OS 2200 Software

ClearPath Extension Kit for OS 2200 User's Guide

Level 3R1

ClearPath OS 2200 Release 22.0

June 2026

8225 2461-003

NO WARRANTIES OF ANY NATURE ARE EXTENDED BY THIS DOCUMENT. Any product or related information described herein is only furnished pursuant and subject to the terms and conditions of a duly executed agreement to purchase or lease equipment or to license software. The only warranties made by Unisys, if any, with respect to the products described in this document are set forth in such agreement. Unisys cannot accept any financial or other responsibility that may be the result of your use of the information in this document or software material, including direct, special, or consequential damages.

You should be very careful to ensure that the use of this information and/or software material complies with the laws, rules, and regulations of the jurisdictions with respect to which it is used.

The information contained herein is subject to change without notice. Revisions may be issued to advise of such changes and/or additions.

Notice to U.S. Government End Users: This software and any accompanying documentation are commercial items which have been developed entirely at private expense. They are delivered and licensed as commercial computer software and commercial computer software documentation within the meaning of the applicable acquisition regulations. Use, reproduction, or disclosure by the Government is subject to the terms of Unisys' standard commercial license for the products, and where applicable, the restricted/limited rights provisions of the contract data rights clauses.

Contents

Section 1. Introduction

1.1.	About this Guide	1-1
1.2.	Audience	1-1
1.3.	Documentation Reference	1-2

Section 2. Using OS 2200 Native Control Programs

2.1.	OS 2200 Native Control Programs	2-1
2.1.1.	ONCP Command Format	2-1
2.1.2.	Open Source Programming Languages and Utilities.....	2-4
2.2.	Differences between OS 2200 and x86-64 Architectures	2-4
2.2.1.	Base System Architecture	2-4
2.2.2.	Operating Systems	2-5
2.2.3.	File System and File Naming.....	2-6
2.2.4.	User Identity and File Ownership	2-9
2.2.5.	File Permissions	2-10
2.2.6.	Command Structure.....	2-10
2.2.7.	Standard Streams	2-10
2.2.8.	Terminal Emulation.....	2-11
2.2.9.	Text File Formats	2-13
2.2.10.	Programming	2-14
2.2.11.	Redirecting Input and Output.....	2-15
2.2.12.	Signaling to Scripts and Programs	2-15

Section 3. Operational Considerations

3.1.	Use of Privileged Operations	3-1
3.2.	Resource Constraints	3-2

Section 4. C Program Interface

Section 5. OS 2200 Native Control Programs (ONCPs)

5.1.	Provided ONCPs.....	5-2
5.1.1.	The @bash Command.....	5-2
5.1.1.1.	Foreground and Background Processes	5-3
5.1.2.	The @git Command	5-3
5.1.3.	The @grep Command	5-4
5.1.4.	The @java and @javac Commands	5-5
5.1.5.	The @nx Command	5-6
5.1.6.	The @pyadd Command	5-6

5.1.7.	The @python and @python3 Commands	5-7
5.1.8.	The @svn Command	5-8
5.2.	Compound ONCP Commands	5-9
5.2.1.	The @nxlink Command	5-9
5.2.2.	The ssg.py Module	5-13
5.2.3.	The exec2200 Command	5-15
5.3.	Linux Tools	5-16

Section 6. Error Messages

Section 7. Limitations

7.1.	Inaccessible CIFS root directories (/bin, /lib, /lib64, /run, /sbin)	7-1
------	---	-----

Tables

5-1. OS 2200 Native Control Programs5-1

Section 1

Introduction

The ClearPath Extension Kit for OS 2200 provides an additional software execution environment for OS 2200 systems. This execution environment allows many applications developed originally for UNIX and Linux execution environments to run mostly unmodified in the highly secure OS 2200 environment. Refer to the *ClearPath OS 2200 Software ClearPath Extension Kit for OS 2200 Conceptual Guide* (8225 2495) for an understanding of the components and communications involved in providing this environment.

This guide describes how to use the provided OS 2200 Native Control Programs (ONCPs) to execute Native Extension processes, including developing new applications using language and scripting processors, such as Python®, Java™, and bash, and executing existing utilities and applications imported from open source repositories. Refer to the *ClearPath OS 2200 Software ClearPath Extension Kit for OS 2200 Installation and Administration Guide* (8225 2446) for a description of how to download and install additional open source applications for use in the Native Extension environment.

1.1. About this Guide

This guide describes how to use the ClearPath Extension Kit for OS 2200. This includes support for modern programming and scripting languages, such as Python, Java™, and bash, and utility programs and applications written for Linux environments.

Note: *This document might have more recent updates, and the latest version is available on the product support site. Enter the part number of the document to search and view the document.*

<https://www.support.unisys.com/common/epa/DocumentationLibraries.aspx?PLA=OS2&NAV=OS2>

1.2. Audience

This guide is intended for application developers and product deployment specialists of OS 2200 systems who wish to employ ClearPath Extension Kit capabilities. Users and developers experienced with both OS 2200 and Linux behavior will benefit from the information in this document.

1.3. Documentation Reference

Refer to the following documents for more information:

- *ClearPath OS 2200 Software ClearPath Extension Kit for OS 2200 Installation and Administration Guide* (8225 2446)
- *ClearPath OS 2200 Software ClearPath Extension Kit for OS 2200 Conceptual Guide* (8225 2495)
- *ClearPath OS 2200 Software Executive Control Language (ECL) and FURPUR Reference Manual* (7830 7949)
- *CIFS for ClearPath OS 2200 Software CIFS User, Programmer, and Administrator Reference Manual* (7859 6137)
- *Application Development Programming Guide* (7831 4077)
- *OS 2200 Universal Compiling System (UCS) Conceptual Overview* (7831 0801)

Note: *The Linux, GNU, and additional vendor-provided open source tools that are the basis for ONCPs are documented extensively elsewhere. URLs for such documentation are provided in each respective section. These URLs were correct and accessible at the time of publication of this document, but may change.*

Section 2

Using OS 2200 Native Control Programs

The ClearPath Extension Kit for OS 2200 provides application programming interfaces (APIs) for interacting with Extension Kit processes (see Section 4) and OS 2200 Native Control Programs (ONCPs) that use the APIs for commonly used program development applications. Each issuing ONCP triggers creation of a process in the Extension Kit execution environment, and sets up communication paths that allow input and output flow between the new process and the ONCP.

This section describes how to invoke and use the supplied ONCPs. In addition, cursory command usage descriptions are provided for open source applications included with this product, but for which ONCP processor calls are not provided. For information on the behavior of specific ONCPs, see Section 5.

2.1. OS 2200 Native Control Programs

For several of the most common programming languages and utilities provided with this product, ONCPs are also provided to allow direct execution in demand runs, runstreams, and addstreams. These ONCPs are based on commonly used open source and software development tools available in UNIX and Linux based systems. Additional details are provided in the following sections.

2.1.1. ONCP Command Format

The provided ONCPs must be called using unformatted ECL. The format is

```
@'<ONCP>[,D] [oncp-options] [input-parameter-1 [input-parameter-2 ... ]]
```

where the provided <ONCP> values are

- BASH
- GIT
- GREP
- JAVA
- JAVAC
- NX
- PYTHON

- PYTHON3
- SVN

Additional ONCPs can be built using the provided tools described in section 4.

The processor call option `D` enables additional logging using each provided ONCP. For each invocation of the ONCP, the log messages are written to a separate file whose name has the form `/tmp/<userid>/<oncp-name>.log`.

The allowed `oncp-option` values for each ONCP are described in section 5. In general, each of these are given as a series of letters or numbers, each preceded by a `"-` or `--`". Multiple `oncp-option` values are separated by spaces.

The allowed input parameters are text strings, separated by spaces, usually signifying input or output file names or ONCP-specific keywords describing operation modes. Depending on the specific ONCP, multiple strings may need to be enclosed in single or double quotes to insure that all the input parameters are communicated to the newly created process.

When the input parameters are file names, each appears in the syntax of an absolute or relative path name. A path name is a series of alphanumeric directory or file names, separated by forward slash characters (`"/`) indicating the location of the file in the file hierarchy provided by CIFS.

Absolute path names start with a single slash, and provide the entire file location specification. Relative path names do not start with a slash, and implicitly start with the Current Working Directory (CWD). Before executing the ONCP command, the CWD can be set using the `cd` or `chdir` command in the CIFS utility (`@CIFSUT`). If not explicitly set, the CWD defaults to the project-id (`/os2200/<project-id>/`).

Successive invocations of ONCPs each restart with the CWD established by the CIFS utility, regardless of any current directory changes made within the Native Extension process launched by an ONCP.

ONCP Usage Modes

Several of the ONCPs have multiple usage modes that affect what options are accepted and how the input parameters are specified. There are three general usage modes for ONCPs: immediate execution, script execution, and interactive execution. The general rules for these three modes are described in the following sections.

Immediate Execution

In immediate execution mode, the ONCP control statement specifies a single execution of a single program in the Native Extension environment. Some ONCPs do not support script or interactive modes, therefore, immediate execution is the only mode available. The supplied ONCPs that support only immediate execution include `@'GREP` and `@'SVN`.

The `oncp-options` you provide are passed to the executing ONCP as they would be if provided when operating on a Linux system. Thus, using the `oncp-option: -c` in the command `@'GREP -c xyz *.py` will search all the Python source files in the current directory for the string `xyz`, but only prints the number of instances of the string `xyz` in each file, rather than the entire matching lines.

The `@'NX` command supports immediate execution of programs (both programs provided by this product, and user-provided programs) in the Native Extension environment for which dedicated processor call lines are not available. The `@'NX` command can also be used to invoke script or interactive execution modes for programs that support these usages, but for which individual processor calls are not available.

It is also possible to implement site-specific ONCPs to avoid the need to use the `@'NX` command by using the interfaces provided with the ClearPath Extension Kit. See section 4 for details.

Script Execution

In script execution mode, the ONCP control statement specifies a script source file, rather than a program to execute. This is supported only by ONCPs that support source scripting, including `@'BASH` and `@'PYTHON`.

The `oncp-options` you provide are passed to the scripting program as command options, as they would be used in a Linux command execution. Thus, the `-c` option (to read an entire command line as the main input parameter) and the `-x` option (echo input lines) are valid options to the `@'BASH` command. Similarly, the `-walways` option can be used to enable all warning messages when using the `@'PYTHON` command.

Interactive Execution

Some ONCPs, including `@'BASH` and `@'PYTHON` support interactive execution mode. In this mode, the `input-parameter` need not be provided. Each ONCP itself issues prompts, accepts subcommands in its supported language, and issues standard output and error messages resulting from the execution of each subcommand.

In this mode, `oncp-options` are still allowed, such as the `-x` option for echoing subcommand lines during `@'bash` execution.

When used in a runstream or addstream, ONCP subcommands can be supplied to ONCPs operating in interactive mode as normal batch input lines.

Since ONCPs trigger execution of widely used open source applications, Unisys does not provide complete documentation for their usage. It is normally provided in the form of manual pages, written by the open source developers, and tutorial information written by the developers and a wide community of users. Refer to the Linux website for more information on manual pages for the most common Linux applications, including most of the utilities provided by the ClearPath Extension Kit.

2.1.2. Open Source Programming Languages and Utilities

In addition to ONCPs, several commonly used programming languages and utilities are provided in the ClearPath Extension Kit. These utilities include

- a C compiler (gcc command)
- a C++ compiler (g++ command)
- GNU Core Utilities
- curl (a utility for downloading files from the web)
- GNU Find Utilities
- GNU Awk (a powerful language for editing text files)
- gzip, zip, and unzip (commands for compressing and uncompressing files)
- GNU Make (a popular software build management tool)
- javac (the compiler for translating Java source code into bytecode)
- rpm (command for creating, modifying, and unpacking software distribution packages)
- sed (a common line editing command)
- tar (a file packaging utility)

Each of these commands and utilities can be invoked from subcommands within @bash, or directly using the @'NX processor call.

Note: As with the supplied ONCPs, Unisys does not include usage documentation for these commands. Refer to the relevant documentation from the original providers and the open source user community.

2.2. Differences between OS 2200 and x86-64 Architectures

OS 2200 and Linux systems can perform all common computing tasks, such as file input and output, accepting interactive or scripted commands, and behaving as servers and clients in networks. However, many of the details are different. When using the Native Extension environment to execute Linux commands and utilities, these differences become apparent.

This section explains and highlights some of the differences, and provides additional resources to allow more effective usage of the ClearPath Extension Kit.

2.2.1. Base System Architecture

Over time, system designs have evolved greatly to run faster, use virtual memory to address large amounts of physical memory, quickly access large permanent storage systems, and have extensive support for networked communications. Many significant differences between the OS 2200 and x86-64 architectures become visible when developing applications and passing data between the environments.

When programs execute in the OS 2200 system architecture, they generate and interpret data files and programs according to that system architecture. When programs, such as Python applications, execute in the Native Extension environment, they generate and interpret data files and programs according to the x86-64 system architecture. Since the binary layouts for most data types (text, integer, floating point, etc.) are different between the two systems, any data passed between the two systems must undergo conversion operations.

The modern version of OS 2200 system architecture is described in the *ClearPath OS 2200 Instruction Processor Programming Reference Manual* (3850 7802). Also, refer to the Intel x86-64 architecture documentation for details.

Portable languages, such as Python and Java are said to be high level because they hide the differences between system architectures. But older programming languages, such as the C language that allow programmers to access the underlying system state and instructions can show different behavior for the same program when using different systems. Therefore, they are considered low level languages. Assembly languages, such as MASM on OS 2200 and gas on Linux, allow direct access to the actual instructions implemented for a given processor architecture, and are thus the lowest “level” programming languages. They allow the most direct control of the processor, but are portable only between processors with the same system architecture.

The following table provides some of the differences between these base system architectures.

Feature	OS 2200 Choice	x86-64 Choice
Memory addressability	36-bit words	Individual 8-bit bytes
Addressing method	Segmented into banks	Flat address space
Text data format	6-bit Fielddata or 9-bit ASCII	8-bit, 16-bit, or 32-bit Unicode
Bit addressing	Big endian	Little endian
Instruction size	Fixed width (36-bits)	Variable byte width
Register addressability	As fixed memory addresses	Not addressable as memory
Integer arithmetic	Ones’ complement binary	Two’s complement binary
Floating point	36 and 72 bits	32, 64, and 128 bits
Vector registers	No	Yes
Indirect instruction execution	Yes	No

2.2.2. Operating Systems

OS 2200 systems run an evolving form of the Exec operating system. The x86-64 execution environment accessible from the ClearPath Extension Kit ONCPs uses a variation of the Linux operating system kernel that, in turn, evolved from the original portable UNIX operating system. While many user-visible features of these two systems are similar, such as support for multiprocessing and processes with multiple

threads, virtual memory support, and support for permanent storage media, such as files on disk, and networking, all the low-level details are different.

Refer to the following documents for more details.

- *Executive Control Language (ECL) and FURPUR Reference Manual* (7830 7949)
- *Exec System Software Operations Reference Manual* (7831 0281)
- *Exec System Software Executive Requests Programming Reference Manual* (7830 7899)
- *ClearPath OS 2200 Software Exec System Software Administration Reference Manual* (7831 0323)
- *Exec System Software Common Banks Programming Guide* (7833 3481)
- *ClearPath OS 2200 Data Structures Reference Manual* (7833 3481)
- *CIFS for ClearPath OS 2200 User Programmer and Administrator Reference Manual* (7859 6137)
- *Exec System Software Executive Control Language (ECL) Programming Quick Reference Guide* (7830 9945)

The Linux kernel and its common programming tools, such as those provided by GNU are open source. Refer to relevant documentation from the related website for details.

Some of the differences between the two architectures that are visible when using the ClearPath Extension Kit are described in the following subsections.

2.2.3. File System and File Naming

The differences between files and file naming is important in the Native Extension environment because files are the main vehicle for conveying application data within and between systems. Binary data types, such as integer, floating point, and structures written into files on the OS 2200 will not be readable without a conversion step in the Native Extension environment, and vice versa. Even the text files conveying ASCII data require conversion because of the different character bit sizes, although this is usually handled automatically as part of network data transmission.

Both of these systems support the existence of files to store and allow writing and reading of sequences of related text or binary records. The description is simplified from these sources to allow easy comparing and contrasting these storage and naming systems.

A quick summary of the different properties of these two systems is provided in the following table.

System Property	OS 2200 File System	CIFS Hierarchical File System
Structure	Flat – Structure in MFD	Hierarchical, rooted at /
File names	qual*filename(C).eltname/version	/dir1/dir2/dir3/. . ./filename

System Property	OS 2200 File System	CIFS Hierarchical File System
Maximum file size	30 gigawords	8.5 gigawords
Name case sensitivity	Insensitive	Insensitive, preserving
Name character set	A:Z, 0:9, - (dash), \$ (dollar sign)	0x01:0xFF except / (slash)
Maximum name length	12 characters (qual, file, elt, ver names)	4095 characters (dir<n> and filename)
Multiple file versions	Supported	Not supported
Temporary names	Yes (@USE and "use" command in CIFSUT)	No, other than symbolic links
Quotation of names	Not necessary	Needed for non-ASCII characters
Names assumed	qual, elt, ver	Prefix (current working directory)
Device names	Outside naming structure	Inside hierarchical structure

To create a file that is easily accessible from both systems, the name must be less than 12 characters, use the OS 2200 supported character set for the names, and construct the file in one of the following forms.

Equivalent OS 2200 Filename	Equivalent CIFS Hierarchical Name
my*fname	/os2200/my/fname
my*fname.eltname/version	/os2200/my/fname/eltname.version

Within the @CIFSUT utility and the execution of ONCPs, such as @bash complete file specifications are given by path names. Path names are sequences of directory names separated by slashes. An absolute path name starts with a slash and provides the complete path through the hierarchy to specify the desired file.

When referring to files in the Native Extension environment, the CWD can be used to provide a relative path name rather than an absolute path to a file. The cd command of the @CIFSUT utility can be used to set the CWD to a specific directory before invoking an ONCP. The ONCP will then start executing using that CWD, and relative path names (omitting the initial slash) can be used to navigate the file hierarchy.

When changing the CWD within a Native Extension process launched by an ONCP, such as the `cd`, `pushd`, or `popd` commands within `@bash`, the new CWD value will continue to be used in subsequent commands within that Native Extension process and any child processes it creates. However, the new CWD value will be lost when that ONCP's Native Extension processes exit. The original CWD in effect when the ONCP was invoked will be restored.

Relative path names provide only the directory names needed to navigate the hierarchy to the desired file. Thus, if the CWD is currently set to `/home/<username>/subdir1/subdir2/`, then using the relative path name `subdir3/myfile.txt` will be equivalent to specifying the absolute path `/home/<username>/subdir1/subdir2/subdir3/myfile.txt`. Additional shortcuts are provided to more conveniently access commonly used relative directories.

- The character `"."` (period) refers to the CWD. The most common use of this syntax is to specify execution of a user program similar to the `@bash` subcommand.
- The sequence `".."` (two periods together) refers to the parent directory. This syntax allows a script to refer to files or different subdirectories above the CWD in the file hierarchy.
- Multiple consecutive slashes are automatically collapsed to a single slash. This syntax allows the CWD specification to end with a slash, while relative path names that use it also provide slashes to indicate subdirectories.

Using these path name syntax facilities, a user or script can employ a relative path name to specify any known location within the current file hierarchy without the knowledge of the absolute path. For example, if the CWD is currently set to `/home/<username>/subdir2/subdir3a/` and the `subdir2` directory also included a subdirectory named `subdir3b`, then a script executing in `subdir3a` could refer to a file in `subdir3b` using the relative path name `../subdir3b/knownfile.txt`, without knowing the name of the parent directory of `subdir3a`. These syntax facilities are used for scripting in the Native Extension environment.

For files that reside in a location other than the first three levels under `/os2200` in the CIFS file hierarchy, such as `/home/<username>/myfile.txt`, the path name cannot be expressed directly as parameters to traditional ECL commands. The `USE` command in the `@CIFSUT` utility can be used to overcome this problem. This command allows you to temporarily associate an OS 2200-usable name (an `@USE` name), with a directory in the CIFS hierarchy. Thus, in the preceding example filename, you can use the following command sequence in a runstream:

```
@CIFSUT
USE MYHOME /home/<username>
@EOF
@PRT MYHOME.MYFILE/TXT
```


Files in the first two file system hierarchy levels under the top level directory `/os2200` are directly accessible by referencing them using the OS 2200 file naming convention (refer to the preceding table of equivalent file names). This requires that files at one or two levels below `/os2200`, such as

```
/os2200/<qualifier> or  
/os2200/<qualifier>/<filename>
```

must follow the OS 2200 file naming rules, that is, a maximum of 12 characters, with characters limited to alphanumeric, and '\$' and '-'.

The Native Extension environment provides a home directory with the absolute path name `/home/<username>` for each individual user. When bash is interpreting paths, the symbol tilde (`~`) can be used as a shortcut name for this directory within a relative path name. Thus, assuming you do not change your CWD using CIFSUT ahead of time, if you issue the command `'cd ~'` within the `@bash` execution environment, your default directory will switch from `/os2200/<project-id>` to `/home/<username>` until you exit `@bash`. Files under the `/home` directory and other top level directories are not directly referable as OS 2200 file names, and thus follow the CIFS naming rules instead.

For a complete description of file naming on OS 2200 systems, refer to the *ECL and FURPUR Reference Manual* (7830 7949). A complete description of the POSIX-like hierarchical directory structure supported by CIFS, as used by the ClearPath Extension Kit product is described in the *CIFS for ClearPath OS 2200 User, Programmer, and Administrator Reference Manual* (7859 6137).

2.2.4. User Identity and File Ownership

OS 2200 provides a rich collection of user identity management and resource ownership facilities. These allow each site to tailor the level of user access and file security appropriately. The default Linux execution environment provides a more limited selection of such tools, revolving around a numeric user identifier (uid) and group identifier (gid), together with a collection of resource access rules using these values for one or more user names and fixed resource permissions ("modes"). Refer to the Linux website for more information.

These two identity management and resource protection environments are not directly equivalent. Nevertheless, CIFS provides numerical uid and gid values for files and associates these values with OS 2200 userids to facilitate normal access and usage in the OS 2200 execution environment. Refer to the *CIFS for Clearpath OS 2200 User, Programmer, and Administrator Reference Manual* (7859 6137) for additional details.

In the Native Extension environment, a new Linux uid / gid pair is supplied to the process as part of each ONCP startup. These values come from a pool and are recycled after each ONCP terminates. They are not maintained across ONCP executions. To view the current uid / gid values in an `@bash` session, issue the commands `id -u` to display the current uid and `id -g` to display the current gid. If a given OS 2200 user activates one or more additional ONCPs while an earlier one is still running, each one after the first will be given the same uid / gid values as the first, and these values will continue to hold until the last ONCP from that user terminates.

During ONCP execution, CIFS uses the OS 2200 userid credentials to determine whether a given file can be accessed, rather than the Linux uid / id and mode values associated with the current ONCP execution. Within an ONCP, such as @bash, file ownership values, such as those displayed by the `ls -l` command show the same numeric values as used within CIFSUT.

Note: *Unisys does not recommend using or writing applications whose behavior depends on specific uid or gid values to govern user process behavior or access to files in the Native Extension environment.*

2.2.5. File Permissions

File permissions can be changed within the Native Extension environment. However, there are slight differences when the change takes effect on an OS 2200 system. In Linux, changing the mode does not affect any process that already has the file open, but does affect all the subsequent open files. In OS 2200, changing the mode does not have any effect until the file assign count is zero, even for assignments that occur after the mode change but while any run has the file assigned.

2.2.6. Command Structure

The syntactical structure of commands given in the OS 2200 execution environment is called the Executive Control Language (ECL). This is specified and the list of available commands and their respective syntax is provided in the *Executive Control Language (ECL) and FURPUR Reference Manual* (7830 7949). Commands are preceded by one or two of the "@" character (also called a "masterspace" in the OS 2200 documentation). Sequences of such commands can be given interactively in demand sessions or included in files and executed as runstreams or addstreams as executable scripts.

Similarly, the syntactical structure of commands given in the Native Extension environment is called the shell command language. In UNIX or Linux terminology, a shell is a utility program, provided as a user interface to the operating system kernel, to interpret commands interactively or in executable scripts. The original shell was designated sh, and script files written in this language are designated with the file type suffix .sh. The primary shell used to support openSUSE Linux and the ClearPath Extension Kit is called bash. The bash shell supports both the command syntax supported by the original sh shell, and an enhanced syntax with easier programming features.

The @bash ONCP is provided by the Native Extension environment to launch an instance of the bash shell from OS 2200. Command usage details for @bash are provided in section 5.

2.2.7. Standard Streams

During execution of OS 2200 runstreams, addstreams, and demand runs, input of commands and text data is taken from script files and terminals or network terminal emulators. Similarly, in UNIX, Linux, and the Native Extension environment, these functions are handled by the standard streams, stdin for input text, stdout for output text, and stderr for error or status messages. The term stream, in this context,

represents a sequence of records, each of variable length that begins when the process or session begins, and only ends when the script ends, or an exit command is given, or the user logs off.

During execution of an OS 2200 runstream, addstream, or demand run, input commands are echoed and output text data is written to a print file or an output terminal. The standard streams behave similarly, and in both cases, controls exist to temporarily divert the input or output streams to a different source or destination.

Because the standard streams are routed through network interfaces as part of ONCP execution, the text undergoes character conversion as appropriate. For stdin, characters are converted from 6-bit Fielddata or 9-bit ASCII (or whatever alternate representation has been configured for a given system) on the OS 2200 side to 8-bit Unicode (or whatever alternate LC_* character settings are in effect) on the SAIL side, and vice-versa for stdout and stderr. The language settings on both the sides must match.

During execution of ONCPs, the standard streams can be redirected using shell operators or programmed I/O operations. The UNIX and Linux program execution environments support this concept to allow for easy composition of operations and scripts within a single interactive command.

In the Native Extension environment, the setup for ONCP execution connects stdin to the input side of runstreams, addstreams, or terminal emulators as appropriate. Similarly, the stdout and stderr streams are connected to the normal OS 2200 stream printing facilities, allowing redirection to files with @BRKPT commands. Since there is only one output text stream in the OS 2200 execution environment, both stdout and stderr streams are combined into a single output stream.

2.2.8. Terminal Emulation

In a demand run on OS 2200, input typed on an attached terminal or a terminal emulator is transmitted via network protocol to the active program to interpret the text strings entered. Similarly, output from the system is sent back through the network to the terminal. A similar process occurs when Linux systems are accessed interactively.

The different usage histories of OS 2200 systems versus UNIX/Linux systems have given rise to different terminal communications protocols being commonly used for the two types of systems. Historically, OS 2200 systems use a screen handling protocol called UTS (for Universal Terminal System) and block-mode transmission protocol called INT1 while UNIX and Linux systems use a screen handling protocol derived from early hardwired terminals, such as vt100, vt220 or ANSI, over various one-character-at-a-time transmission protocols. Both sets of protocols offer key usage features. When a given terminal emulator behavior matches the protocols expected by the server, these features work well. However, when the terminal emulator screen or transmission protocol has a mismatch with the server's expectation, unexpected behavior is likely.

A problem arises when an attempt is made to use a character-at-a-time emulator for a system that only accepts block mode behavior or vice-versa. When using a UTS

emulator with a demand run on an OS 2200 system, all is well due to the UTS-aware communication handlers within OS 2200. Similarly, a terminal emulator using Telnet transmission protocol, such as PuTTY, accesses a Linux system functions as expected. In the Native Extension environment, these two systems are necessarily mixed.

It is possible to log into an OS 2200 demand session with a UTS terminal emulator, such as InfoConnect, which offers UTS-specific features. It is also possible to log in to a demand session using a full-featured Telnet emulator, such as PuTTY, however, most character-based features are lost in this environment, since characters are not delivered to the executing program until the enter key is pressed.

The executables accessible through ONCPs were developed for use with Telnet and other character-at-a-time protocols. Consequently, when accessed via a demand session using a UTS emulator, all character-at-a-time features are lost because the UTS protocol does not support them. When accessed via a Telnet-based emulator, all character-at-a-time features are lost because the OS 2200 terminal handler cannot interpret them and are not passed along to the ONCP.

Thus, while both types of terminal emulator allow entering commands to interactive ONCPs, such as `@python` and `@bash`, commonly used keying features, such as command recall, command history, filename completion, etc., are not available. Some non-standard settings, either in the emulator or as commands within an ONCP can slightly improve the user experience as shown in the following table.

The following table depicts terminal emulator behavior when using interactive ONCPs.

Terminal Emulator Type	Settings and Features
UTS, such as InfoConnect	Normal for OS 2200 demand usage. Within ONCP: Backspace erases characters within an input line. Mouse-based copy-paste works as expected. Within <code>@bash</code> , use the <code>stty</code> command to set window size according to the actual screen size, such as <code>stty rows 48</code> <code>stty cols 132</code>

Terminal Emulator Type	Settings and Features
Telnet, such as PuTTY	PuTTY settings - Terminal: Local echo: Force On - Terminal: Local line editing: Force On - Keyboard: Backspace key: Ctrl-H Backspace erases characters within an input line. Mouse-based copy-paste works as expected. Command recall does not work (keys misinterpreted). Filename completion using <tab> characters completes, but does not display completed characters back on the emulator. Within @bash, Use the <code>stty</code> command to set window size according to the actual screen size, such as <code>stty rows 48</code> <code>stty cols 132</code>

2.2.9. Text File Formats

ASCII text files represented as System Data Format (SDF) files are normally compatible with ordinary text files when used inside ONCPs as command or data files. Since CIFS processes all files moving between the two environments, its data storage behavior is relevant. Refer to the *CIFS for ClearPath OS 2200 User, Programmer, and Administrator Reference Manual* (7859 61360) for details.

The Fielddata coding system, historically important on OS 2200 systems is not recognized by any ClearPath Extension Kit utility programs or ONCPs as command or input text data or generated as output. CIFS automatically converts any script or data files or elements represented using Fielddata characters to ASCII characters before passing them to a Native Extension process launched by an ONCP, and always generates ASCII format output.

To facilitate data access by programs using traditional OS 2200 I/O mechanisms, CIFS interprets various line termination sequences as record breaks when generating files as follows:

- A single line feed (0x0A) – the Linux/UNIX standard compatible with UCS Runtime System I/O handling
- A carriage return line feed sequence (0x0D and 0x0A) – used by Windows text files
- A single carriage return (0x0D) – the normal Apple MacOS line terminator

If you use a Windows text file as a shell script, such as an input script for @bash, it will result in error messages due to the extraneous carriage return character at the end of each line.

Files using terminators other than a plain line feed can be filtered to make them compatible with Linux/UNIX based programs by using the `w`-option of the CIFSUT `cp` and `copy` commands.

These line termination sequences are commonly found when downloading the files, using CIFS directly or utilities, such as `curl` in the Native Extension environment, or using Windows-specific editors, such as Notepad.

2.2.10. Programming

Both the historically supported OS 2200 languages, such as the ones supported by the Unisys Universal Compiling System (UCS) and many traditional languages used by the open source community, use the traditional compile-link-run sequence to prepare programs for execution. Refer to the *Application Development Programming Guide* (7831 4077) and the *OS 2200 Universal Compiling System (UCS) Conceptual Overview* (7831 0801) for details on the programming environments supported on the OS 2200.

In the open source community, languages intended for high performance execution, such as C, FORTRAN, and C++ use similar steps to generate efficient executable code, although the compilers object file formats, and utilities such as debuggers, are all different from OS 2200. Open source compilers for the C and C++ languages are included with the ClearPath Extension Kit. For open source programming languages, the compile and link steps can be combined into a single command, and the executable output file can be copied anywhere the proper run-time libraries are present.

For modern languages that emphasize portability and ease-of-use features rather than high performance, a translation step from source code into a portable representation called a bytecode is performed instead. The bytecode representation is then executed directly by a run-time interpreter. Such interpreters include advanced features, such as garbage collection and direct interfaces to system features. Both the Java and Python programming environments work this way. For Java, the source-to-bytecode translation is performed by a separate compilation step, called `javac`, prior to executing the bytecode form. For Python, the Python interpreter handles the translation to bytecode automatically as a preliminary execution step.

In all cases, extensive run-time libraries are needed to support common operations that need to interface with the system. For the C language, only a few modest size libraries, such as `libc` are needed. For the C++ language, the `libstdc++` library is needed as well, and any of dozens of special purpose libraries if extensions or system interfaces are used. For both Java and Python, system specific and application specific libraries have been developed for use by the open source community.

There are too many such libraries available to allow them all to be included with the ClearPath Extension Kit. However, most widely published libraries are available from open source package repositories, such as openSUSE and GitHub, and can be downloaded and installed within the Native Extension environment. Refer to the *ClearPath OS 2200 Software ClearPath Extension Kit for OS 2200 Installation and Administration Guide* (8225 2446) for instructions on downloading and installing additional packages.

2.2.11. Redirecting Input and Output

Output from programs and scripts can be routed to files for later printing or review. On OS 2200, this output redirection and reversion is accomplished using the @BRKPT command. Similarly, input redirection can be done with the @ADD command. For more information, refer to the *Executive Control Language (ECL) and FURPUR Reference Manual* (7830 7949).

Within scripts, programs, and interactive commands used within the Native Extension environment, redirection of both input and output files is accomplished with the I/O redirection system built into the basic I/O system and incorporated into the command syntax of command shells, including @bash.

Briefly, to specify a file as an alternate source of input for a command, use the < character (less-than sign), followed by the file name of the desired input file. To specify a file as the desired destination for the output of a command, use the > character (greater-than sign), followed by the desired file name. Similarly, to redirect error messages from a command, use the sequence &> (ampersand followed by greater-than sign), followed by the desired output file name. To append to an existing file, rather than creating a new file, use the >> sequence (two greater-than signs) instead. To pipe the output of one command for use as input to another command on the same command line, use the | (vertical bar) character to separate the two commands. To efficiently dispose of command output without saving it, send it explicitly to the "null" device, using the syntax > /dev/null.

Once you are familiar with the available commands and these redirection methods, it is possible to use these shell syntax elements as a programming tool, especially for text file manipulation, both in scripts and interactively. For the redirection capabilities of @bash, refer to the GNU documentation and for the special characters usable in @bash commands, refer to documentation at <https://www.tldp.org>.

2.2.12. Signaling to Scripts and Programs

The normal command to use at the end of shell command session, including @bash, is `exit`.

Programming language interpreters, such as Python have explicit commands for exiting (`quit()` for Python).

Due to issues with terminal emulation as described above, single characters that are available for exiting, interrupting, or signaling end-of-input, when executing programs or scripts in the Linux context, such as ^C (0x03), ^D (0x04), ^X (0x18), ^Q (0x11), ^S (0x13), or ^Z (0x1A) may not be effective in the Native Extension environment.

Therefore, the Native Extension environment provides these supplemental signaling methods:

- To signal the 'end of input' condition in the stdin stream, use the same sequence as when signaling the end-of-input in an ECL runstream: a record containing the string @EOF. When running in a demand session, under some terminal emulators, entering a `control-D` followed by a <CR> also works.

- Input commands that begin with an @ (master space) character are immediately interpreted as ECL commands and thus cause the current ONCP session to end. The only exceptions are the @EOF command, which signals end-of-input, and commands that begin with @@ (double master space), which are interpreted as transparent ECL commands.
- When working interactively in a demand session, use the command @@X O, followed by a <CR> to interrupt a runaway output operation. This may require a few seconds to take effect, due to the details of output buffering, but will eventually cause the ONCP to discard all remaining output until another input is made.
- In a demand session, in some terminal emulators, the sequence `control-C` followed by <CR> will stop the running program and return control within the Native Extension environment.
- In a demand session, in some terminal emulators, the sequence `control-Z`, followed by a <CR>, will stop the running program but keep it active as a stopped task in the background, while returning terminal input control. You can see all stopped jobs by issuing the `jobs` command. To continue a stopped job, issue the command `fg <jobnumber>`. To permanently kill background jobs, issue the command `kill -9 $(jobs -p)`. The jobs will enter the killed state, and be removed after entering an additional command.
- In a demand session, to immediately interrupt and stop an ONCP, use the command @@X T<CR>. This will normally result in returning to ECL in control mode; an @@END command is necessary to restore normal scrolling operation.

Section 3

Operational Considerations

While the ClearPath Extension Kit for OS 2200 provides support for a large portion of the Linux execution environment, not all existing Linux programs and scripts will work as expected. Usually, failures are due to the following operational restrictions.

3.1. Use of Privileged Operations

The Native Extension environment is not the same as a running Linux system. For system stability and security, it operates only as a set of containerized, unprivileged processes. Programs running in this environment cannot become privileged and have no access to system resources that would allow privileged operations. When attempting to port existing applications from a direct Linux execution environment, such as SUSE or RedHat Linux, built-in attempts to access privileged services will fail. To get such applications to work, alternative source coding techniques that do not involve use of privileges are needed. Some of these non-functional privileged operations are

- **SuperUser.** User accounts on Linux systems provide a set of permissions that allow the user to perform system administrative tasks. User accounts that have activated these permissions are called superusers. No user accounts on OS 2200 systems are equivalent to Linux superuser accounts within the Native Extension environment. System administrator privileges on OS 2200 systems are required to configure and modify the behavior of Native Extension environment. Therefore, traditional Linux shell commands needed to activate privileged behavior, such as logging into an account named *root*, or using the *su* or *sudo* commands in *bash*, are not functional in this environment.
- **User-specific files.** CIFS provides each user with a subdirectory under */home*, which is visible to other users, but whose contents are by default not accessible to any other userid. CIFS also provides a */tmp* directory in which any user can create, modify, and delete files, but, unlike */home*, files in */tmp* are visible and accessible to other users by default. To prevent other users from accessing or deleting your files under */tmp*, use the *chmod* command in *@CIFSUT* or *@BASH* to control such access.

- **Devices.** On systems, the `mount` and `umount` commands support mounting and unmounting devices and file systems. These commands are not provided in the Native Extension environment. CIFS provides the `/dev/null`, `/dev/random`, `/dev/tty`, and `/dev/urandom` devices as a convenience for Native Extension processes with their normal semantics. The `mount` and `umount` commands may be used under CIFSUT with appropriate privileges. Some utilities and programs attempt to provide a user prompt, such as a password request by opening additional devices under `/dev`, but this is not currently supported. Examples are `sftp` and `ssh`.
- **The `/proc` file system.** The Linux kernel provides a variety of services to allow processes to query and modify system behavior via a synthetic file system under the directory `/proc`. Each time a process references an entry in this directory hierarchy, the Linux kernel intercepts the I/O request and satisfies it either via synthesizing the requested information (for input requests) or updating a system behavior parameter (for output requests). Such requests will only succeed if the requesting process has sufficient privilege. In the Native Extension environment, CIFS maintains a directory named `/proc`, and it provides a read-only subset of the entries from the SAIL `/proc` hierarchy. Information about specific Native Extension processes can only be accessed by the OS 2200 userid that started those processes.
- **Shared memory.** The Linux kernel provides a set of features to allow processes to share pages of virtual memory for communication and performance. Attempting to use any of these features in a process running under the Native Extension environment will fail.
- **Namespaces and control groups.** The Native Extension environment is implemented using the namespaces and control groups (cgroups) features of the SAIL kernel, and thus these features cannot be used within Native Extension processes.

The Linux kernel provides a wide variety of privileged capabilities. These privileged capabilities are not available within the Native Extension environment.

The Native Extension environment runs within a containing SAIL system. However, due to the containment, aspects of the SAIL system that require privilege are not available within the Native Extension environment.

3.2. Resource Constraints

Processes running in the Native Extension environment are constrained with respect to system resources. Some resource limits pertain to individual active ONCPs, while others pertain to the aggregate of all ONCPs running at a specific time. Limits can be adjusted by the system administrator. When a resource request exceeds one of these limits, the relevant low-level system service call will return a failure status, which will sometimes become visible to the user as a failure within the running application, depending on the specific programming choices in that specific application.

Many applications can accommodate such failures without issuing user visible notifications. For example, system memory limitations can be overcome by storing data in permanent storage more frequently. Others can result in fatal errors at the

time they occur, and will generally be reflected in specific error messages emitted to the stderr stream from the application.

The global resource limitations imposed by the Native Extension environment are

- Number of simultaneously active ONCP sessions – default: 10
- Amount of system CPU usage available to operating ONCPs – default: 100% of 1 CPU core. ONCPs do not run on the same system CPU core as an OS 2200 Instruction Processor. Therefore, depending on the system configuration, there may be less than 100% of one CPU core actually available for executing Native Extension processes.
- Amount of working system memory available – default: 10%
- Number of simultaneously active user tasks (threads) – default: 1000
- Core file size – 0 bytes.

The per-ONCP invocation resource limitations imposed by the Native Extension environment are

- Accumulated CPU time from all tasks running on all cores – default: 86,400 seconds (1 day)
- Number of independent processes simultaneously active – default: 100
- Amount of working system memory available – default: 1 GiB

The additional resource limitations imposed by the Native Extension environment that cannot be changed by the system administrator are

- Maximum file size. The normal limits for OS 2200 files are applied by CIFS.
- Number of pending signals – 251957
- Maximum amount of locked memory – 64 KiB
- Pipe buffer size – in 512 byte units – 8
- Message queue size – 819200 bytes
- Stack size – 8192 KiB

The OS 2200 system console command `NPC STATUS` can be used to display the current resource limitation settings. Refer to the *ClearPath OS 2200 Software ClearPath Extension Kit for OS 2200 Installation and Administration Guide* (8225 2446) for instructions on how to modify these settings.

If the system administrator reduces one or more limit values while an ONCP is active, the new limits will not impact the running session, but will apply to new ONCPs executed subsequently.

Since some common system resources are used for both the base OS 2200 system and any processes activated by ONCP commands, the system administrator may adjust these limits up or down depending on the operational requirements of each

Operational Considerations

individual system. The initial default values shown here only apply to the current version of the ClearPath Extension Kit product and will likely change in future versions.

Section 4

C Program Interface

This section describes the C Application Program Interfaces (APIs) supplied by the NPCLIB component. User programs written in UC can use the NPCLIB APIs by including the following function definition statement:

```
#include <npclib.h>
```

`_native_start`

Use the `_native_start` function to start executing a Native Extension process.

Synopsis

```
void* _native_start(native_args* args_packet)
```

Description

The `_native_start()` function allocates and initializes a program-level native control area used by all NPCLIB functions and starts a process in the Native Extension environment. NPCLIB environment attributes and process options are passed to NPCLIB in the `args_packet` parameter.

Field Name	Type	Description
version	unsigned short	The packet version number. The <code>npclib.h</code> <code>#define</code> <code>NATIVE_PKT_VERSION</code> provides the value of the highest supported version.
arg_count	unsigned short	The number of arguments present on the process command line.
modes	unsigned int	A bitmask used to specify one or a combination of NPCLIB modes. 0001 NX_CALLER_IO 0002 NX_OUTPUT_ANSI 0004 NX_PRINTFILE
target	char*	The path to the process to start.
targ_args	char**	The process command line arguments.

Field Name	Type	Description
debug_file	void*	A pointer to an object returned by <code>_cifs_fopen()</code> or NULL. Note: By passing a valid pointer, the caller specifies that NPCLIB functions will write trace information to the file.
screen_rows	unsigned short	(Packet version 2 or later.) Specifies the number of display rows for demand and TIP sessions. If zero, NPCLIB attempts to automatically determine the screen rows for UTS terminals.
screen_cols	unsigned short	(Packet version 2 or later.) Specifies the number of display columns for demand and TIP sessions. If zero, NPCLIB attempts to automatically determine the screen columns for UTS terminals.
app_group	unsigned int	(Packet version 3 or later.) Specifies the application group to use for TIP transactions.
mcb_api	function pointer	(Packet version 3 or later.) Specifies the entry point for the desired MCB to use in a TIP transaction. This is set automatically when NXLINKE creates an ONCP with the <code>-g</code> option.

NPCLIB Modes

- `NX_CALLER_IO` By default, NPCLIB creates input and output handlers to transfer data between SYMB\$ or an MCB and the Native Extension process. Control is only returned to the caller upon process termination. Set this mode to cause `_native_start()` to return immediately after the process is started. The caller can then use the `_native_read()` and `_native_write()` APIs to send and receive data.
- `NX_OUTPUT_ANSI` By default, the NPC Daemon converts Native Extension process output from ANSI/XTERM format to UTS. Set this mode to allow output through without conversion, such as when using PuTTY in a demand session.

Return Values

If it completes successfully, the `_native_start()` function returns a pointer the native control area used by NPCLIB. Otherwise, it returns a null pointer and sets `errno` to identify the specific error.

The native control area pointer must be included on every call to an NPCLIB function.

Error Codes

For the following conditions, `_native_start()` returns `NULL` and sets `errno` to the corresponding value:

EBACKNOTACTIVE	Either the CIFSB or NPCBG background run is not active when it is required.
EINVAL	An invalid value was detected in a <i>native_args</i> packet field.
ENOENT	The target program specified in the <i>native_args</i> packet was not found.
ENOMEM	Allocation of the native control area failed.
ENOTTY	An error was encountered starting the target process.
EPIPE	An error occurred communicating with the Native Extension process.

Note: Additional information about specific errors may be available if the Native Extension process was started with a *debug_file* specified in the *native_args* packet.

`_native_read`

Use the `_native_read()` function to read data from a process running in the Native Extension environment.

Synopsis

```
int _native_read(void* native_handle, char* str, int len, int mode)
```

Description

The `_native_read()` function reads data from the process started by the `_native_start()` call. If no data is available after thirty seconds, control is returned to the caller.

native_handle

is a pointer to the native control area returned by `_native_start()`.

str

is a pointer to the buffer that receives the data.

len

is the number of bytes to be read.

mode

is not supported yet and should be set to 0.

Return Values

If the `_native_read()` function completes successfully, the number of bytes actually read and placed into `str` is returned. Otherwise, `_native_read()` sets `errno` to the specific error and returns -1.

Error Codes

EAGAIN	No data was available after thirty seconds. A subsequent call might return data.
EINVAL	An invalid <i>native_handle</i> was detected.
EPIPE	An error occurred communicating with the Native Extension process.

Note: Additional information about specific errors may be available if the Native Extension process was started with a `debug_file` specified in the `native_args` packet.

`_native_write`

Use the `_native_write()` function to write data to a process running in the Native Extension environment.

Synopsis

```
int _native_write(void* native_handle, char* str, int len, int mode)
```

Description

The `_native_write()` function writes data to the process started by the `_native_start()` call.

native_handle

is a pointer to the native control area returned by `native_start()`.

str

is a pointer to the buffer holding the data.

len

is the number of bytes to be written.

mode

is not supported yet and should be set to 0.

Return Values

If the `_native_write()` function completes successfully, the number of bytes actually sent to the process is returned. Otherwise, `_native_write()` sets `errno` to the specific error and returns -1.

Error Codes

EINVAL	An invalid <i>native_handle</i> was detected.
EPIPE	An error occurred communicating with the Native Extension process.

Note: Additional information about specific errors may be available if the Native Extension process was started with a *debug_file* specified in the *native_args* packet.

`_native_stop`

Use the `_native_stop()` function to terminate the execution of a process running in the Native Extension environment.

Synopsis

```
int _native_stop(void* native_handle)
```

Description

The `_native_stop()` function stops the process started by the `_native_start()` function and deallocates the program-level native control area.

native_handle

is a pointer to the native control area returned by `native_start()`.

Return Values

If it completes successfully, the `_native_stop()` function returns 0. Otherwise, it returns a -1 and sets *errno* to identify the specific error.

Error Codes

EPIPE	An error occurred communicating with the Native Extension process.
-------	--

Note: Additional information about specific errors may be available if the Native Extension process was started with a *debug_file* specified in the *native_args* packet.

Section 5

OS 2200 Native Control Programs (ONCPs)

This section describes the use of and gives specific information about the individual OS 2200 Native Control Programs (ONCPs). Using the lower level detailed capabilities of these facilities is provided by normal open source documentation, such as programming language tutorials, manual pages, and message boards, and are described separately for each ONCP. Currently, the ONCPs provided with the ClearPath Extension Kit are detailed in the following table.

Table 5–1. OS 2200 Native Control Programs

ONCP	Description
@bash	The UNIX/Linux interactive and scripting utility
@git	The widely used version control system
@grep	The file contents searching utility
@java	The Java portable programming environment
@nx	General command for launching executables that do not have a dedicated ONCP
@python	Python interpreter
@svn	An instance of the Subversion code management system
@javac	The Java programming language compiler
@pyadd	Use a python application to generate and execute an ECL script
@nxlink	Package a Native Extension application as an OS 2200 executable program (a ZOOM object)

ONCPs can easily be built using programming tools and templates provided with the Native Extension environment.

5.1. Provided ONCPs

The OS 2200 Native Control Programs (ONCPs) described in this section are direct analogs of the equivalent Linux commands that you can issue from bash or other command shells. For each, if no parameters used, then a simple masterspace ("@") can be used to introduce the ONCP. But if additional parameters are needed, use unformatted ECL (masterspace followed by an apostrophe @').

5.1.1. The @bash Command

The Bourne Again Shell (Bash) is a Linux shell and command language that executes commands from standard input or a file. It is suitable for issuing interactive commands, such as file manipulation and for complex scripting. It is widely used as the tool for performing software builds and testing. Detailed bash usage documentation is available at <https://www.gnu.org/software/bash/manual/>.

The @bash command invokes the bash shell in the Native Extension environment. The formats for the command are:

Format 1: @'BASH[,D] [bash-options] [cifs-format-filename]

Format 2: @'BASH[,D] -c "command-string"

Both formats allow you to provide options and script filenames with the syntax and semantics as given in the bash documentation.

Use Format 1 if you have only a single input parameter, that is, a script file, for bash to execute in non-interactive mode. In this format, input parameters that appear after the cifs-format-filename are ignored.

Use Format 2 if you are invoking bash with a complex command with multiple elements, such as to specify a script file with one or more parameters. Use the bash escape character syntax described in the bash manual to specify special or literal characters within the command-string, such as \n to specify a newline character.

In either Format

- The command option `D` requests the generation of debugging output showing the commands used to show invocation and execution details used in the OS 2200 Native Extension environment.
- The `bash-options` are a list of single character options preceded by a `-` (dash) character, or one or more string options preceded by `--` (a double dash). Use the commands `@'bash --help` and `@'bash -c "help set"` for a description of the available `bash-options`.
- If no filename or command-string is given, bash is invoked as an interactive shell. In this mode, characters are passed along from the terminal driver as individual commands for bash to execute, until an exit command is given or is interrupted by an @@X ECL command. Output and error strings written to stdin and stderr are written back to the terminal as they occur.

Normally, the `@bash` command generates terminal control characters according to the protocol specifications of the kind of terminal currently in use. The `B` option instructs the ONCP to suppress this special character processing, and treat all output as if it is being generated into a breakpoint file, rather than a terminal.

5.1.1.1. Foreground and Background Processes

Each command issued within `@bash`, whether interactively or within a script, causes a subprocess to be created within which to run the command. This subprocess takes over the `stdin` stream, while the `bash` process continues to wait for input in the background. When the command completes, its subprocess terminates and input control returns to `bash` itself. When the original `@bash` command exits, by any means, all subprocesses also terminate whether they are stopped or still running.

From within `@bash`, you can launch a command into a background subprocess directly by appending the `&` character (ampersand) to the end of the command. The `@bash` process will keep control of `stdin` and you can continue to issue `bash` commands. If a background subprocess requests input, it will stop and wait until it is brought into the foreground with a `fg` command. The `jobs` command allows you to see the status of all background jobs.

If the terminal emulator supports it, you can also stop a running foreground subprocess by issuing the command sequence `^Z <CR>`. This sequence returns control of the `stdin` stream of commands to `bash` itself. This allows you to either kill the job or let it to continue with an `fg` command. To kill it, use the `jobs -p` command to find its process id, then using the `kill <pid>` command.

Meanwhile, both `bash` itself, and any subprocesses running in the background can output to the `stdout` and `stderr` streams. That output will appear on an interactive terminal or a print file, depending on how `@bash` is invoked.

Since all subprocesses terminate when the original `@bash` command exits, trying to put a subprocess into the background, by using an `&` at the end of the `@bash` command itself, will fail. The subprocess will terminate immediately when the `@bash` command exits, before it has a chance to perform any actions.

5.1.2. The @git Command

Git is the name of a source code management tool widely used within the open source software development community. It allows even widely dispersed contributors to collaborate on a single set of source code files (or other documents, such as contracts or drawings) without interfering with one another's work or losing track of important contributions. Detailed git usage documentation is available at <https://git-scm.com/>.

The Native Extension environment offers the `@git` command to invoke the installed git utility. This allows users to clone a copy of another user's repository, fetch changes from a remote repository, and merge your changes with others without the need to start a `bash` shell. Because this command requires options and multiple input parameters, only one command format is useful.

Format: `@'GIT[,D] git-subcommand [parameter1 [parameter2 ...]]`

The command option `D` requests the generation of debugging output showing the commands used to show invocation and execution details used in the Native Extension environment.

The `git-subcommands` provide the main action to be performed by the `git` ONCP. The parameters used are different for each subcommand. Once you are familiar with the general workings of `git`, use the help commands such as `@'git help` and `@'git help <git-subcommand>` for a quick refresher on the proper syntax.

`Git` employs standard URL syntax for referring to such repositories, which is supported in the Native Extension environment. For example, to clone a set of sources on a remote system, you might use the following command:

```
@'git clone https://github.com/<owner>/<repository>.git
```

This will allow future source file fetches and merges to refer to this remote repository without the need to always specify its network location.

Normally, the `@git` command generates terminal control characters according to the protocol specifications of the kind of terminal currently in use. The `B` option instructs the ONCP to suppress this special character processing, and treat all output as if it is being generated into a breakpoint file, rather than a terminal.

5.1.3. The `@grep` Command

The `grep` command is a powerful text-searching tool originally invented for UNIX and has been included in all UNIX and Linux distributions. Detailed `grep` usage documentation is available at <https://www.gnu.org/software/grep/manual/grep.html>.

The Native Extension environment provides the `@grep` command to invoke `grep` directly from a demand or batch session. Because `grep` requires two or more parameter specifiers (the search string and the target file(s)), only one command format is useful.

Format: `@'GREP[,D] [grep-options] <search-string> [target-file1 [targetfile2 ... ]]`

The command option `D` requests the generation of debugging output showing the commands used to show invocation and execution details used in the Native Extension environment.

The `grep` command offers useful options to modify the search criteria. Use the command `@'grep --help` for a description of the `grep-options` available.

The `search-string` can be a simple sequence of alphanumeric characters to search for in the target files, or an arbitrarily complex regular expression. If the `search-string` contains white space, enclose the entire search string in apostrophes or quotes to pass the search string along to the `grep` command intact.

For example, to search for the phrase `new file` in all the log files in the CWD, issue the command: `@'grep "new file" *.log`.

The `target-files` may be any sequence of CIFS file names that are acceptable to `bash`, including wildcards. Using the `grep-option` of `-r` (search recursively) and a `target-file` of `*` (all files in the CWD) is a powerful way to search for a string in an entire directory and all its subdirectories.

Normally, the `@grep` command generates terminal control characters according to the protocol specifications of the kind of terminal currently in use. The `B` option instructs the ONCP to suppress this special character processing, and treat all output as if it is being generated into a breakpoint file, rather than a terminal.

5.1.4. The `@java` and `@javac` Commands

Java is a widely used portable programming language whose bytecode-based implementation allows programs to run on a wide variety of devices. Detailed Java programming and command information can be found at <https://openjdk.org/>.

The Native Extension environment provides Java support in the form of the commands `@javac` to compile Java programs into bytecode, and `@java` to execute Java applications.

Compile one or more Java source files as follows:

```
@'JAVAC[,D] [ javac-options ] [ sourcefiles ] [ @argfiles ]
```

This command reads class and interface definitions, written in the Java programming language, and compiles them into bytecode class files. The command option `D` requests the generation of debugging output showing the commands used to show invocation and execution details used in the Native Extension environment. For a refresher on the available `javac-options`, issue the command `@'javac -help`. To compile a small number of source files, enter their names separated by whitespace on the command line directly. For a large number of source files, list the file names in a file separated by blanks or line breaks. Then use the list file name on the `@'javac` command line, preceded by an `@` character—one or more `@argfiles`. The documentation and man page describe additional conventions to follow when using the `@'javac` command.

To run the resulting Java class file, you can issue the `@java` command from a demand session as follows:

```
@JAVA[,D] [ java-options ] <cifs-classfile-path>
```

The command option `D` requests the generation of debugging output showing the commands used to show invocation and execution details used in the Native Extension environment.

The class file name is given in CIFS format even when the apostrophe does not precede the command name.

Normally, the `@java` and `@javac` commands generate terminal control characters according to the protocol specifications of the kind of terminal currently in use. The `B` option instructs the ONCP to suppress this special character processing, and treat all output as if it is being generated into a breakpoint file, rather than a terminal.

5.1.5. The `@nx` Command

The `@nx` command is provided by the Native Extension environment as a convenience to specify any single Linux command, even if no equivalent ONCP has been provided. For example, to display the CWD issue the command `@'nx pwd`.

The `@nx` command has the format:

```
@'NX[,D] linux-command [opts-or-parameter [parameter . . .]]
```

Use `@'nx` to invoke a single Linux command with or without options or parameters. For example, the command `@'nx ls -l mycompile.log` requests that bash give a long-format listing of the file `mycompile.log` in the current working directory.

Because the `@nx` command bypasses the bash shell, no wildcard expansion takes place, so using any command that depends on the use of wildcards must force the use of bash to execute the target command. For example, the command `@'nx bash -c "ls -l *.log"` requests a long format listing of all files in the current directory whose names end in the string `".log"`.

The command option `D` requests the generation of debugging output showing the commands used to show invocation and execution details used in the Native Extension environment.

Normally, the `@nx` command generates terminal control characters according to the protocol specifications of the kind of terminal currently in use. The `B` option instructs the ONCP to suppress this special character processing, and treat all output as if it is being generated into a breakpoint file, rather than a terminal.

5.1.6. The `@pyadd` Command

The Native Extension environment provides the `@pyadd` command to make it possible to use a python program to generate and run arbitrary ECL addstreams. This command operates in two steps:

1. Run an arbitrary python script to generate an output file
2. Use the output file as the input to an `@ADD` command

A typical use for the `@pyadd` command is to generate a build script for a multiple-module software program or package.

The `@pyadd` command has the format:

```
@'pyadd [-l] [-n] [-o <output-file>] <python.script> [args . . .]
```


Option	Description
-l	Specifies that @pyadd should do an @ADD,L rather than just @ADD (that is, include the commands in the listing).
-n	Causes @pyadd to suppress step 2—generate the @ADD file but do not run it.
-o	Allows you to specify the path for the generated file. The default path is /tmp/<userid>/py\$add/py\$add.add.

The output file path is passed to the python script as its first argument. Any additional arguments provided on the @pyadd command are passed to the python script, in order, following this first argument.

Normally, the @pyadd command generates terminal control characters according to the protocol specifications of the kind of terminal currently in use. The B option instructs the ONCP to suppress this special character processing, and treat all output as if it is being generated into a breakpoint file, rather than a terminal.

5.1.7. The @python and @python3 Commands

Python is a family of closely related programming languages that is widely used by the open source community and for which a large variety of user-provided libraries are available. In operation, Python resembles the bash shell, in that it can be used either for interactive input from a terminal emulator, or to execute script files. Detailed Python language and command documentation can be found at <https://www.python.org/doc/>.

The Native Extension environment provides a Python version 3 interpreter using the command @python3. The environment also provides an @python command, that is synonymous with @python3.

The @python command invokes the python interpreter. The formats for this command is:

Format 1: @'PYTHON[,D] [python-options] [cifs-filename-path]

Format 2: @'PYTHON[,D] -c "command-string"

Both formats allow you to provide options and script filenames with the syntax and semantics. Refer to the Python documentation for details.

Use Format 1 if you have only a single input parameter, that is, a script file for Python to execute in non-interactive mode. In this format, input parameters that appear after the cifs-format-filename are ignored.

Use Format 2 if you are invoking Python with a complex command with multiple elements, such as to specify a script file with one or more parameters. Use the bash escape character syntax described in the bash manual to specify special or literal characters within the command-string, such as \n to specify a newline character.

Normally, the `@python` and `@python3` commands generate terminal control characters according to the protocol specifications of the kind of terminal currently in use. The `B` option instructs the ONCP to suppress this special character processing, and treat all output as if it is being generated into a breakpoint file, rather than a terminal.

In either Format

- The command option `D` requests the generation of debugging output showing the commands used to show invocation and execution details used in the Native Extension environment.
- The `python-options` are a list of single character options preceded by a `-` (dash) character. Use the command `@'python --help` for a description of the available options.
- If no filename or command-string is given, Python is invoked in interactive mode. In this mode, characters are passed along from the terminal driver as individual commands for Python to execute, until an `@eof` signals end-of-file or is interrupted by an `@@X` ECL command. Output and error strings written to `stdin` and `stderr` are written back to the terminal as they occur.

5.1.8. The `@svn` Command

Subversion is the name of a source code management tool widely used within the open source software development community. It allows even widely dispersed contributors to collaborate on a single set of source code files (or other documents, such as contracts or drawings) without interfering with one another's work or losing track of important contributions. Detailed subversion usage documentation can be found at <https://subversion.apache.org/docs/>.

The Native Extension environment offers the `@svn` command to invoke the installed Subversion ONCP. This allows users to checkout, update private copies, and contribute changes back to local or remote source code repositories without the need to start a bash shell. Because this command typically requires options and multiple input parameters, only one command format is useful.

```
@'SVN[,D] svn-subcommand [parameter1 [parameter2 ...]]
```

The command option `D` requests the generation of debugging output showing the commands used to show invocation and execution details used in the Native Extension environment.

The `svn-subcommands` give the main action to be performed by the Subversion processor. The parameters used are different for each subcommand. Once you are familiar with the general workings of Subversion, use the help commands, such as `@'svn help` and `@'svn help <svn-subcommand>` for a quick refresher on the proper syntax.

It is common for the repository of a given project to reside on a different system from the user. Subversion provides a custom syntax for referring to such repositories, which is supported in the Native Extension environment. For example, to checkout a set of sources on a remote system, you might use a command of the form:

```
@'svn co svn://system7.irv.mycorp.com/home/project3/svnrepos/
development/ proj-tools new-tools
```

This will allow future source file updates and checkins to refer to this remote repository without the need to always specify its network location.

Normally, the @svn command generates terminal control characters according to the protocol specifications of the kind of terminal currently in use. The B option instructs the ONCP to suppress this special character processing, and treat all output as if it is being generated into a breakpoint file, rather than a terminal.

5.2. Compound ONCP Commands

The ONCPs described in this section are not direct analogs of any existing Linux commands. Instead, they encapsulate sequences of operations using the Native Extension environment to accomplish tasks directly useful in the OS 2200 environment.

5.2.1. The @nxlink Command

The @nxlink command provides the ability to package one or more Native Extension commands into a single OS 2200 executable object module (a ZOOM file) so that it may be executed directly as an OS 2200 program. When executed, the generated ZOOM launches the specified Native Extension process as if it was one of the standard ONCPs, together with whatever arguments are on the @nxlink command line, plus any additional ones from the command line invoking the ZOOM execution. An example usage would be to specify one of the ONCP language processors, such as Python, Java, or Bash, plus an input user program and its parameters on the @nxlink command line. The command format for @nxlink is:

```
@'NXLINK[,H] [-a] [-b] [-d] [-g<n>] [-i<p>] [-l] [-o <output_zoom>]
<nx_program> [<static_options>] [<static_args> . . .]
```

Use the H-option with @nxlink to display a help screen describing all of the options and parameters.

Option	Description
-a	Specifies the use of ANSI rather than UTS terminal handling in nx_program.
-d	Specifies the use of debug mode when executing nx_program.
-g<n>	Specifies that the output ZOOM should be compiled and linked for the given decimal application group number. This allows the nx_program to be deployed as a TIP transaction.
-i<p>	In the generated ZOOM, include the content of the file path given as the specified parameter number, rather than leave it as a parameter for the ZOOM.
-l	Produce long listings for the compilation and link of the generated ZOOM.

Option	Description
-o	Specifies the path name for the generated ZOOM file. If a ".abs" name extension is not given, one is appended. If the -o option is omitted, or just a directory is provided, the ZOOM name is taken from the <code>nx_program</code> or the name of the program specified by the -i parameter.
<nx_program>	Use this program as the primary executable for beginning execution of the generated ZOOM. If the specified executable is not located in the current directory nor in any of the default search path directories (/usr/local/bin:/usr/bin:/bin), the full path must be specified.
<static_options>	Option arguments (that is, with a "-" prefix) to pass to the <code>nx_program</code> each time the ZOOM is run. When the generated ZOOM is executed in batch or demand, this option, and the <code>static_args</code> , are passed directly to <code>nx_program</code> when the generated ZOOM is executed.
<static_args>	The remaining arguments that are passed to <code>nx_program</code> each time the ZOOM is executed.

The `nx_program` argument specifies either a standalone compiled program that can run successfully in the Native Extension environment, or one of the provided language processors, such as Python, Java, or bash. In the latter case, the first `static_args` argument is used to specify which user program to run, after one or more `static_options`, such as `-m` (for Python), `-jar` (for Java), or `-c` (for Python or bash). In any of these cases, the first of the `static_args` specifies the path name for the user program to run.

The `@nxlink` command runs several system commands, which issue status and progress messages. Actual compilation and link listings can be generated by using the `-l` option.

In the generated ZOOM, `@nxlink` normally provides only a path name reference to the `nx_program` as a parameter on the command line. At execution time, the generated ZOOM finds the specified file and sets up the execution environment for it automatically. This allows the user of the generated ZOOM to run it on the OS 2200 system where it was generated, or any other OS 2200 system where the ClearPath Extension Kit is installed and the specified program or script copied into an identically named directory. The purpose of the `@nxlink -i` option is to remove this dependency on the location of the program by including the specified program code directly into the generated ZOOM. At most one such program can be included, so selecting the user program or script for such treatment is the normal case. By choosing to include the user program or script in this way, the generated ZOOM will be portable to any OS 2200 system with the ClearPath Extension Kit installed. In this way, the `@nxlink` command acts as a packaging and distribution aid for OS 2200 applications utilizing the ClearPath Extension Kit.

If you use the `-i` option without the `<p>` parameter number, the `@nxlink` command infers which program to include by looking at the values specified for `nx_program`, `static_options`, and `static_args`. If the `nx_program` value is one of `python`, `java`, or `bash`, then `@nxlink` will select the first `static_args` value for inclusion when `-i` is specified. Otherwise, `@nxlink` chooses to include the file specified by the

`nx_program` parameter itself. In this case, if the first `static_args` argument is not a regular executable file accessible to `@nxlink`, it will issue an error message.

If this default behavior for the `-i` option is inadequate for specifying which program to include, you can use the `-i<p>` form of the option, where the `<p>` value must immediately follow `-i`, with no intervening whitespace. The `<p>` value is the decimal number of the `@nxlink` command line parameter to use. The `nx_program` field is parameter 0, with each `static_options` and `static_args` value sequentially numbered from there.

Normally, the `@nxlink` command produces a ZOOM file whose output includes special character processing for conforming to the appropriate terminal control protocol used when the `@nxlink` command is issued. The `-b` option suppresses this special character processing in the generated ZOOM form of the program. Thus, the output of the ZOOM will be appropriate for formatting into a breakpoint file or print file.

The generated ZOOM will process the `static_options` and `static_args` specified in the `@nxlink` command, in order, at startup time. Any additional parameters given on the command line for executing the generated ZOOM are processed after these.

Examples

```
>@cifsut
CIFSUT 9R2 (2018-Nov-01 11:28:28) 2019-May-02 Thu 20:21:02

>cd /home/username/tests

>ls -l
-rwxrwxrwx 1 username [none] 87 2019/05/02 13:18:06 hello.cpp
-rwxrwxrwx 1 username [none] 133 2019/05/02 17:49:36 HelloWorld.java
-rwxrwxrwx 1 username [none] 35 2019/05/03 16:06:33 hw.py
-rwxrwxrwx 1 username [none] 23 2019/05/02 18:34:40 Manifest.txt

>use td /home/username/tests

>@eof
End CIFSUT 20:23:50
>@'nxlink -o hwbash bash -c "echo 'Hello World from bash!'"
. . . messages . . .
>@td.hwbash
Hello World from bash!
>@'nxlink -o hwpy python hw.py
. . . messages . . .
>@td.hwpy
Hello World from Python!
>@'nxlink -o hwipy -i python hw.py
. . . messages . . .
>@td.hwipy
Hello World from Python!
>@'javac HelloWorld.java
>@'nx jar -cfe HelloWorld.jar HelloWorld HelloWorld.class
>@'nxlink -o hwjava java -jar HelloWorld.jar
```

```

. . . messages . . .
>@td.hwjava
Hello World from Java!
>@'nxlink -o hwijava -i java -jar HelloWorld.jar
. . . messages . . .
>@td.hwijava
Hello World from Java!
>@'nx g++ -o hwcpp hello.cpp
>@'nxlink hwcpp
. . . messages . . .
>@td.hwcpp
Hello World from C++!
>@'nxlink -o hwicpp -i hwcpp
. . . messages . . .
>@td.hwicpp
Hello World from C++!
>@'nx ls -l
total 4932
-rw-r--r-- 1 username [none] 436 May 6 18:12 HelloWorld.class
-rw-r--r-- 1 username [none] 786 May 6 18:06 HelloWorld.jar
-rwxrwxrwx 1 username [none] 133 May 2 17:49 HelloWorld.java
-rwxrwxrwx 1 username [none] 23 May 2 18:34 Manifest.txt
-rwxrwxrwx 1 username [none] 87 May 2 13:18 hello.cpp
-rwxrwxrwx 1 username [none] 35 May 3 16:06 hw.py
-rwxrwxrwx 1 username [none] 713340 May 2 20:24 hwbash..abs
-rwxr-xr-x 1 username [none] 12640 May 6 18:16 hwcpp
-rwxrwxrwx 1 username [none] 711324 May 6 18:16 hwcpp..abs
-rwxrwxrwx 1 username [none] 738204 May 6 18:25 hwicpp..abs
-rwxrwxrwx 1 username [none] 721908 May 6 18:23 hwijava..abs
-rwxrwxrwx 1 username [none] 720564 May 6 18:20 hwipy..abs
-rwxrwxrwx 1 username [none] 713340 May 6 18:13 hwjava..abs
-rwxrwxrwx 1 username [none] 713172 May 3 17:38 hwpy..abs

```

Notes:

- Verify the input file names.
- The CIFS “use” command establishes the shortcut name “td” for ease of referencing the output files.
- In the first example, @nxlink is used to generate a ZOOM named td.hwbash directly from an inline bash command, and verifies correct execution.
- In the second example, @nxlink is used to generate ZOOM td.hwpy from a Python input file in two ways (by reference and by inclusion of hw.py), and these are tested.
- In the third example, @javac is used to compile HelloWorld.java, then an @nx command is used to generate a corresponding Java archive file. Then @nxlink is used to generate executable ZOOM files in two ways, and tested.
- In the fourth example, an @nx command is used to compile a C++ program and an @nxlink is used to generate an executable ZOOM in two ways, and these are tested.

- After all these commands, the generated ZOOM files are present, and the versions using the include option are larger.

5.2.2. The ssg.py Module

OS 2200 provides a powerful stream manipulation processor called Symbolic Stream Generator, in the form of the @SSG command. Refer to the *Symbolic Stream Generator Programming Reference Manual* (7831 1784-009) for details. The Symbolic Stream Generator accepts its control input in the form of Stream Generation Statements (SGSs). Production software builds in the ECL environment often use SGSs to specify a variety of related information, such as source and destination files, command options, and specific commands. The Native Extension environment includes a Python module named `ssg.py` that provides a parser for SGSs. After importing the `ssg.py` module, you can call the parsing function `ssg.parse_sgs(<sgs_input_file_spec>)`. The `ssg.parse_sgs()` function parses the lines in the specified file, and generates an output variable named `ssg.sgs` of type `defaultdict(list)` that contains the parsed lines as a dictionary of keyword, list pairs. Each keyword corresponds to the label used on one or more of the SGS input lines, and each list corresponds to the values provided on the remainder of the line. Each list may, in turn, be divided into sublists of items separated with commas within each field of the SGS input lines.

Example 1

For the following example SGS file:

```
>@'nx cat ssgexample_315.sgs
Where 183,Adomain      Mpls      MN      56634
When  21,October,2005  +12465
Who   Cable,Able      Where,1    When,2
Where 154,Robert      St Paul   MN      55127
When  12,April,2005   564Z,,LAST
Who   Brian,Juliet    Where,2    When,1
```

The parser generates the following output, as shown by the following interactive Python sequence:

```
>@python
Python 3.7.3 (default, Apr 09 2019, 05:18:21) [GCC] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import ssg
>>> ssg.parse_sgs("ssgexample_315.sgs")
>>> ssg.sgs
defaultdict(<class 'list'>, {'where': [[['183', 'Adomain'], ['Mpls'],
['MN'], ['56634']], [['154', 'Robert'], ['St'], ['Paul'], ['MN
'], ['55127']]], 'when': [[['21', 'October', '2005'], ['+12465']],
[['12', 'April', '2005'], ['564Z', '', 'LAST']]], 'who': [[['Cabl
e', 'Able'], ['Where', '1'], ['When', '2']], [['Brian', 'Juliet'],
['Where', '2'], ['When', '1']]])
```

The `ssg.py` module provides a powerful tool in the ECL environment for specifying input, output and control parameters for software builds and other tabular data processing.

Example 2

This can be used as a replacement or alternative to SSG. The training exercise shows two functionally equivalent scripts—one using SSG, and one using Python. The goal is to look through product installs within `SYSS$*DATA$.CO$INSTALL$/COMUS$` and find all installation files for each install of a given product (in this case, RDMS). The functionally equivalent SSG and Python scripts are shown below, as well as their identical output.

SSG

```
@ssg,abke ,sys$*data$.co$install$/comus$
sgs
favorite_product  RDMS
@eof
skel
*set instancenum = 0
*clear we_are_done
*increment a to [install] while we_are_done is clear
  *if column search from install,instancenum+1,1,2 for
  [favorite_product,1,1,1,ucstr$]
    *display 'my favorite product is [install,stmt$,1,2] '
    *set fieldnum = 0
    *clear all_files
    *increment b to [install,stmt$] while all_files is clear
    *if row search from install,stmt$,fieldnum+1,1 for FILE
      *display 'in file [install,stmt$,fld$,2] '
      *set fieldnum = fld$
    *else
      *set all_files
    *endif
  *loop
  *set instancenum = stmt$
*else
  *set we_are_done
*endif
*loop
@eof
@eof
```

Python

```
import sys
import ssg

favorite_product = 'RDMS'

ssg.parse_sgs("/os2200/sys$/data$/co$install$.comus$")
for install in ssg.sgs["install"]:
```



```

prog_name = install[0][1]
if (prog_name == favorite_product):
    print("my favorite product is", prog_name)
for field in install:
    if (field[0] == 'FILE'):
        print("in file", field[1])

```

Sample Output (identical for both SSG and Python)

```

my favorite product is RDMS
in file UDS$$SVN*RDMS(83)
in file UDS$$SVN*RDMSLIB(83)
in file UDS$$SVN*RSA(83)
in file UDS$$SVN*RDMS$CFGSCOD(83)
in file UDS$$SVN*RDMS$CFGSDEF(83)
my favorite product is RDMS
in file SYS$LIB$*RDMS(254)
in file SYS$LIB$*RDMSLIB(250)
in file SYS$LIB$*RSA(243)
in file SYS$LIB$*RDMS$CFGSCOD(214)
in file SYS$LIB$*RDMS$CFGSDEF(214)
my favorite product is RDMS
in file UDS$$FOR*RDMS(3)
in file UDS$$FOR*RDMSLIB(9)
in file UDS$$FOR*RSA(9)
in file UDS$$FOR*RDMS$CFGSCOD(5)
in file UDS$$FOR*RDMS$CFGSDEF(5)
my favorite product is RDMS
in file UDS$$FIV*RDMS(37)
in file UDS$$FIV*RDMSLIB(37)
in file UDS$$FIV*RSA(37)
in file UDS$$FIV*RDMS$CFGSCOD(37)
in file UDS$$FIV*RDMS$CFGSDEF(37)

```

5.2.3. The exec2200 Command

The exec2200 command provides the ability to launch a 2200 demand run and execute the requested 2200 commands from an Extension Kit process. All commands are executed with the same credentials and privileges of the executing user. The demand session remains active until the calling Extension Kit process terminates.

Example 1

This example shows how to execute a @prt command from within @bash. The @prt command is executed and output is displayed without terminating the bash session. Note that when specifying commands with spaces to exec2200, the commands must be surrounded by quotes (for example, exec2200 'cd /tmp').

```

>@bash
~ # exec2200 @prt,il
FURPUR 33R2 (2018-02-12 19:37:41 CST) 2024-05-01 09:19:32
FEST*TPF$(0),SCDISK,T

```

```
FEST*DIAG$(0),SCDISK,T
SYS$*LIB$(1),SCDISK,A
SYS$LIB$*FURPUR(208),SCDISK,A  CSINTNALT$,
```

Example 2

This example shows how to execute a sequence of CIFSUT commands from Python. Note that these commands are executed sequentially and retain context from the previously executed commands. Once the Python process terminates, the demand session will also terminate.

```
>@python
Python 3.6.15 (default, Sep 23 2021, 15:41:43) [GCC] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import os
>>> _ = os.system("exec2200 @cifsut")
CIFSUT 9R3 (2020-Feb-10 15:51:17) 2024-May-01 Wed 09:30:18
>>> _ = os.system("exec2200 'cd /tmp'")
>>> _ = os.system("exec2200 ls")
.dotnet
clr-debug-pipe-11478-1164435-in
clr-debug-pipe-11477-1164435-in      pacman
```

5.3. Linux Tools

The ClearPath Extension Kit provides the main set of programming tools needed to build and run an application originally written for Linux. In addition to the ONCPs, many additional commands and libraries are provided. These include the gnu compilers and libraries for the C and C++ languages, gcc and g++, since the existing compilers for these languages available in the traditional OS 2200 programming environment cannot employ the x86-64 machine structures and instruction set.

Section 6

Error Messages

The Native Extension components involve significant resource management and communication, which may fail in a variety of ways. When a component or message fails while executing an ONCP, the ONCP endeavors to provide an error code and message showing what failed. This section gives a summary of these messages with suggested actions for how to remedy them.

Debugging Output

When using the **D** option on any ONCP, the communication across the interface between NPCLIB, running in the OS 2200 context, and the NPC Daemon, running in the SAIL context, is written into a log file, in addition to triggering their normal behavior. The log file is named `/tmp/<userid>/<oncp-name>.log`, where `<userid>` reflects the user-id that issued the command, and `<oncp-name>` reflects the name of the ONCP that was invoked.

The messages document the behavior in the ClearPath Extension Kit C programming interface (see section 4). The arguments to each call and any resulting output values or error codes are logged.

Additional log files are available to the system administrator to allow configuring and diagnosing Native Extension behavior.

Component Availability

If one of the components providing the Native Extension environment is not communicating or has stopped running, then any ONCP command will fail with a message indicating that condition. The environment needs to be restarted.

Resource Exhaustion

When a resource, such as working memory or available processes is exhausted or would be exhausted in order to complete a given operation, a message signifying that condition will be returned and issued. If this condition is not fatal to the given ONCP, such as **@bash**, then the ONCP will continue to offer services. Otherwise, it will terminate. The remedy is to either modify the Native Extension resource constraint configuration to provide additional resources, or modify the script or program to reduce the resource requirements.

Section 7

Limitations

This section describes the limitation in the OS 2200 Extension Kit.

7.1. Inaccessible CIFS root directories (/bin, /lib, /lib64, /run, /sbin)

The Extension Kit reserves use of the /bin, /lib, /lib64, /run, and /sbin directory names at the CIFS root. All files on the OS 2200 and all other CIFS root directories (for example, /home and /tmp) are accessible by Extension Kit processes. The /bin, /lib, /lib64, /run, and /sbin directories can be created and the files in them can be accessed through CIFS, but they are invisible to the Extension Kit. All Extension Kit processes are unable to see any of these CIFS directories or files in them.

Limitations

